

OPERASI CITRA SEDERHANA

NAMA : Ardhan Dikri Achmad Fahrudin
NIM : 2441070020012
KELAS : TI-3B

1. Setup & Mount Google Drive / Import Library

Jika Anda menjalankan notebook ini di **Google Colab**, sel ini akan otomatis melakukan mount Google Drive. Jika dijalankan di **Lokal**, mount akan di-skip secara otomatis.

[CODE CELL #2]

PYTHON

```
from google.colab import drive
drive.mount('/content/drive')

import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
import glob
import math
import os

# Helper function untuk membaca citra dengan toleransi path lokal/drive
def load_image(path, flags=cv.IMREAD_COLOR):
    if not os.path.exists(path):
        base_name = os.path.basename(path)
        if os.path.exists(base_name):
            path = base_name
        else:
            raise FileNotFoundError(f"File tidak ditemukan di path: {path}")
    return cv.imread(path, flags)
```

Mounted at /content/drive

D1. Operasi Citra Sederhana

Percobaan 3: Transformasi Linier Brightness

Formula: $g(x,y) = f(x,y) + b$

```

print(' Mengubah tingkat kecerahan citra ')
print('_____')
try:
    brightness = int(input('Masukkan nilai kecerahan: '))
except ValueError:
    brightness = 30
    print('Error, not a number. Menggunakan default brightness = 30')

# Path gambar houses.jpg di Drive
img_path = '/content/drive/MyDrive/PCVK/Images/houses.jpg'
try:
    original = load_image(img_path)
except Exception as e:
    print(f"Warning: {e}. Menggunakan citra sintesis dummy.")
    original = np.zeros((300, 400, 3), dtype=np.uint8)
    cv.rectangle(original, (50, 50), (200, 200), (100, 150, 200), -1)

brightness_image = np.zeros(original.shape, original.dtype)

# Akses per piksel 3 loop perulangan (height, width, channels)
for y in range(original.shape[0]):
    for x in range(original.shape[1]):
        for c in range(original.shape[2]):
            brightness_image[y, x, c] = np.clip(original[y, x, c] + brightness, 0, 255)

# Atau menggunakan cara cepat: brightness_image = cv.convertScaleAbs(original, beta=brightness)
final_frame = cv.hconcat((original, brightness_image))

# Display dengan matplotlib
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(cv.cvtColor(original, cv.COLOR_BGR2RGB))
axes[0].set_title('Original')
axes[0].axis('off')

axes[1].imshow(cv.cvtColor(brightness_image, cv.COLOR_BGR2RGB))
axes[1].set_title(f'Brightness Image (+{brightness})')
axes[1].axis('off')
plt.tight_layout()
plt.show()

```

```

Mengubah tingkat kecerahan citra
-----
Masukkan nilai kecerahan: 86

```

Original



Brightness Image (+86)



TUGAS PRAKTIKUM

1. Inverse Citra (Negative Image)

Formula: $g(x,y) = 255 - f(x,y)$

```
img_path = '/content/drive/MyDrive//PCVK/Images/peppers.jpg'
try:
    img_orig = load_image(img_path)
except Exception as e:
    print(f"Warning: {e}. Menggunakan citra dummy.")
    img_orig = np.zeros((300, 400, 3), dtype=np.uint8)
    cv.circle(img_orig, (150, 150), 80, (0, 0, 255), -1)
    cv.circle(img_orig, (250, 150), 80, (0, 255, 0), -1)

# Inverse Citra:  $g(x,y) = 255 - f(x,y)$ 
img_inverse = 255 - img_orig

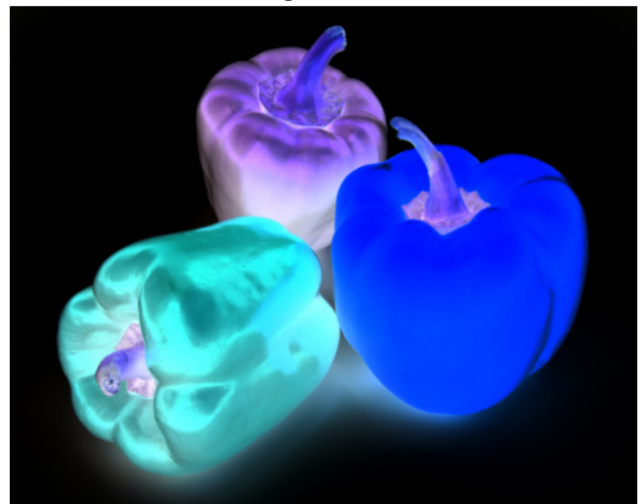
# Tampilkan hasil
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(cv.cvtColor(img_orig, cv.COLOR_BGR2RGB))
axes[0].set_title('Citra Asli (peppers.jpg)')
axes[0].axis('off')

axes[1].imshow(cv.cvtColor(img_inverse, cv.COLOR_BGR2RGB))
axes[1].set_title('Citra Negative (Inverse)')
axes[1].axis('off')
plt.tight_layout()
plt.show()
```

Citra Asli (peppers.jpg)



Citra Negative (Inverse)



2. Transformasi Contrast

Formula Correction Factor:

$$F =$$

$$\frac{259 \cdot (C + 255)}{255 \cdot (259 - C)}$$

Transformasi Piksel:

$$R' = \text{Truncate}(F \cdot (R - 128) + 128)$$

```

print(' Mengubah kontras dan tingkat kecerahan citra ')
print('_____')
try:
    brightness = int(input('Masukkan tingkat kecerahan: '))
except ValueError:
    brightness = 10

try:
    contrast = float(input('Masukkan kontras (misal 1.5): '))
except ValueError:
    contrast = 1.5

img_path = '/content/drive/MyDrive/PCVK/Images/houses.jpg'
try:
    img_orig = load_image(img_path)
except Exception as e:
    print(f"Warning: {e}. Menggunakan citra dummy.")
    img_orig = np.full((300, 400, 3), 100, dtype=np.uint8)

# Konversi nilai contrast multiplier jika perlu
if contrast ≤ 10.0 and contrast ≥ -10.0 and contrast ≠ 0:
    C = (contrast - 1.0) * 128.0
else:
    C = contrast

factor = (259.0 * (C + 255.0)) / (255.0 * (259.0 - C))

img_contrast = np.zeros(img_orig.shape, dtype=np.float64)
for c in range(3):
    img_contrast[:, :, c] = factor * (img_orig[:, :, c].astype(np.float64) - 128.0) + 128.0

img_contrast = np.clip(img_contrast, 0, 255).astype(np.uint8)

fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(cv.cvtColor(img_orig, cv.COLOR_BGR2RGB))
axes[0].set_title('Citra Asli')
axes[0].axis('off')

axes[1].imshow(cv.cvtColor(img_contrast, cv.COLOR_BGR2RGB))
axes[1].set_title(f'Contrast={contrast}, Brightness={brightness}')
axes[1].axis('off')
plt.tight_layout()
plt.show()

```

Mengubah kontras dan tingkat kecerahan citra

Masukkan tingkat kecerahan: 10
Masukkan kontras (misal 1.5): 1.5

Citra Asli



Contrast=1.5, Brightness=10



3. Transformasi Logarithmic Brightness

Formula Log:

$$s = c \cdot \log(1 + r)$$


```
print(' Mengubah tingkat kecerahan citra dengan Transformasi Log ')
print('_____')

img_path = '/content/drive/MyDrive/PCVK/Images/houses.jpg'
try:
    img_orig = load_image(img_path)
except Exception as e:
    print(f"Warning: {e}. Menggunakan citra dummy.")
    img_orig = np.full((300, 400, 3), 30, dtype=np.uint8)

r = img_orig.astype(np.float32)
c = 255.0 / np.log(1.0 + np.max(r))
s = c * np.log(1.0 + r)
img_log = np.clip(s, 0, 255).astype(np.uint8)

fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(cv.cvtColor(img_orig, cv.COLOR_BGR2RGB))
axes[0].set_title('Citra Asli (Gelap)')
axes[0].axis('off')

axes[1].imshow(cv.cvtColor(img_log, cv.COLOR_BGR2RGB))
axes[1].set_title('Transformasi Logarithmic Brightness')
axes[1].axis('off')
plt.tight_layout()
plt.show()
```

Mengubah tingkat kecerahan citra dengan Transformasi Log



4. Transformasi Grayscale (Averaging, Lightness, Luminance)

1. $\text{Grayscale}_{\text{avg}} =$

Averaging:
$$\frac{R + G + B}{3}$$

2. $\text{Grayscale}_{\text{lightness}} =$

Lightness:
$$\frac{\max(R, G, B) + \min(R, G, B)}{2}$$

3. **Luminance:** $\text{Grayscale}_{\text{luminance}} = 0.21 R + 0.72 G + 0.07 B$

```
img_path = '/content/drive/MyDrive/PCVK/Images/peppers.jpg'
try:
    img_orig = load_image(img_path)
except Exception as e:
    print(f"Warning: {e}. Menggunakan citra dummy.")
    img_orig = np.zeros((300, 400, 3), dtype=np.uint8)
    img_orig[:, :133] = [0, 0, 255] # Merah
    img_orig[:, 133:266] = [0, 255, 0] # Hijau
    img_orig[:, 266:] = [255, 0, 0] # Biru

img_rgb = cv.cvtColor(img_orig, cv.COLOR_BGR2RGB).astype(np.float64)
R, G, B = img_rgb[:, :, 0], img_rgb[:, :, 1], img_rgb[:, :, 2]

# a. Averaging
gray_avg = np.clip((R + G + B) / 3.0, 0, 255).astype(np.uint8)

# b. Lightness
max_val = np.maximum(np.maximum(R, G), B)
min_val = np.minimum(np.minimum(R, G), B)
gray_lightness = np.clip((max_val + min_val) / 2.0, 0, 255).astype(np.uint8)

# c. Luminance
gray_luminance = np.clip(0.21 * R + 0.72 * G + 0.07 * B, 0, 255).astype(np.uint8)

fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes[0, 0].imshow(cv.cvtColor(img_orig, cv.COLOR_BGR2RGB))
axes[0, 0].set_title('Citra Asli')
axes[0, 0].axis('off')

axes[0, 1].imshow(gray_avg, cmap='gray')
axes[0, 1].set_title('a. Averaging Grayscale')
axes[0, 1].axis('off')

axes[1, 0].imshow(gray_lightness, cmap='gray')
axes[1, 0].set_title('b. Lightness Grayscale')
axes[1, 0].axis('off')

axes[1, 1].imshow(gray_luminance, cmap='gray')
axes[1, 1].set_title('c. Luminance Grayscale')
axes[1, 1].axis('off')

plt.tight_layout()
plt.show()
```

Citra Asli



a. Averaging Grayscale



b. Lightness Grayscale



c. Luminance Grayscale



5. Selective Color Grayscale (Hanya Warna Merah yang Berwarna)

Warna merah dipertahankan, sedangkan warna lain diubah ke grayscale.

```
img_path = '/content/drive/MyDrive/PCVK/Images/peppers.jpg'
try:
    img_orig = load_image(img_path)
except Exception as e:
    print(f"Warning: {e}. Menggunakan citra dummy.")
    img_orig = np.zeros((300, 400, 3), dtype=np.uint8)
    cv.circle(img_orig, (150, 150), 80, (0, 0, 255), -1)
    cv.circle(img_orig, (250, 150), 80, (0, 255, 0), -1)

# Deteksi warna merah via HSV
hsv = cv.cvtColor(img_orig, cv.COLOR_BGR2HSV)
lower_red1, upper_red1 = np.array([0, 50, 50]), np.array([10, 255, 255])
lower_red2, upper_red2 = np.array([170, 50, 50]), np.array([180, 255, 255])

mask1 = cv.inRange(hsv, lower_red1, upper_red1)
mask2 = cv.inRange(hsv, lower_red2, upper_red2)
red_mask = cv.bitwise_or(mask1, mask2)

gray_single = cv.cvtColor(img_orig, cv.COLOR_BGR2GRAY)
gray_3ch = cv.cvtColor(gray_single, cv.COLOR_GRAY2BGR)

# Gabungkan citra asli (pada mask merah) dengan citra grayscale (pada non-merah)
result = np.where(red_mask[:, :, np.newaxis] > 0, img_orig, gray_3ch)

fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(cv.cvtColor(img_orig, cv.COLOR_BGR2RGB))
axes[0].set_title('Citra Asli')
axes[0].axis('off')

axes[1].imshow(cv.cvtColor(result, cv.COLOR_BGR2RGB))
axes[1].set_title('Red Only (Lainnya Grayscale)')
axes[1].axis('off')
plt.tight_layout()
plt.show()
```

Citra Asli



Red Only (Lainnya Grayscale)



6. Gamma Correction

Formula Power-Law / Gamma:

$$\left(\frac{I}{255} \right)^{\gamma}$$

$$I' = 255 \cdot$$

$$\left(\frac{I}{255} \right)^{\frac{1}{\gamma}}$$

```

print(' Gamma Correction pada citra ')
print('_____')
try:
    gamma_val = float(input('Masukkan nilai Gamma (misal 0.5 atau 3.0): '))
except ValueError:
    gamma_val = 0.5
    print('Error, not a number. Default gamma = 0.5')

img_path = '/content/drive/MyDrive/PCVK/Images/houses.jpg'
try:
    img_orig = load_image(img_path)
except Exception as e:
    print(f"Warning: {e}. Menggunakan citra dummy.")
    img_orig = np.full((300, 400, 3), 100, dtype=np.uint8)

# Hitung Gamma Correction via LUT
inv_gamma = 1.0 / gamma_val
lut = np.array([(i / 255.0) ** inv_gamma) * 255 for i in np.arange(0, 256)]).astype(np.uint8)
gamma_corrected = cv.LUT(img_orig, lut)

fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(cv.cvtColor(img_orig, cv.COLOR_BGR2RGB))
axes[0].set_title('Citra Asli')
axes[0].axis('off')

axes[1].imshow(cv.cvtColor(gamma_corrected, cv.COLOR_BGR2RGB))
axes[1].set_title(f'Gamma Correction ( $\gamma = \{gamma\_val\}$ )')
axes[1].axis('off')
plt.tight_layout()
plt.show()

```

```

Gamma Correction pada citra
-----
Masukkan nilai Gamma (misal 0.5 atau 3.0): 4

```

Citra Asli



Gamma Correction ($\gamma = 4.0$)



7. Simulasi Image Depth (Kuantisasi Citra)

Formula Kuantisasi:

$\text{level} =$

$$\frac{255}{2^{\text{bit_depth}}}$$

$- 1$

$C' = \text{round}$

$$\left(\frac{C}{\text{level}} \cdot \text{level} \right)$$


```
try:
    bit_depth = int(input('Masukkan bit depth tujuan (1-7): '))
except ValueError:
    bit_depth = 3

print(f"Bit depth awal : 8 bit (256 level)")
print(f"Bit depth tujuan : {bit_depth} bit ({2**bit_depth} level)")

level = 255.0 / (pow(2, bit_depth) - 1)
print(f"Jumlah level step: {level:.2f}")

img_path = '/content/drive/MyDrive/PCVK/Images/peppers.jpg'
try:
    original = load_image(img_path, cv.IMREAD_GRAYSCALE)
except Exception as e:
    print(f"Warning: {e}. Menggunakan citra dummy.")
    original = np.linspace(0, 255, 300*400, dtype=np.uint8).reshape((300, 400))

depth_image = np.round(original.astype(np.float64) / level) * level
depth_image = np.clip(depth_image, 0, 255).astype(np.uint8)

fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(original, cmap='gray')
axes[0].set_title('Grayscale 8-bit (Asli)')
axes[0].axis('off')

axes[1].imshow(depth_image, cmap='gray')
axes[1].set_title(f'Grayscale {bit_depth}-bit ({2**bit_depth} level)')
axes[1].axis('off')
plt.tight_layout()
plt.show()
```

```
Masukkan bit depth tujuan (1-7): 7
Bit depth awal : 8 bit (256 level)
Bit depth tujuan : 7 bit (128 level)
Jumlah level step: 2.01
```

Grayscale 8-bit (Asli)



Grayscale 7-bit (128 level)



8. Modul Average Denoising & PSNR

Formula PSNR:

$$\text{PSNR} = 20 \cdot \log_{10} \left(\frac{255}{\sqrt{\text{MSE}}} \right)$$

}}

```

def calc_PSNR(img1, img2):
    mse = np.mean((img1.astype(np.float64) - img2.astype(np.float64)) ** 2)
    if mse == 0:
        return 100.0
    max_pixel = 255.0
    psnr = 20 * math.log10(max_pixel / math.sqrt(mse))
    return psnr

# Load citra galaxy.jpg
galaxy_path = '/content/drive/MyDrive/PCVK/Images/galaxy.jpg'
try:
    orig_galaxy = load_image(galaxy_path)
except Exception as e:
    print(f"Warning: {e}. Menggunakan citra dummy galaxy.")
    orig_galaxy = np.full((300, 300, 3), 50, dtype=np.uint8)
    cv.circle(orig_galaxy, (150, 150), 60, (200, 220, 255), -1)

# Load 100 citra ternoise dari folder noises/*.jpg
noises_path_glob = '/content/drive/MyDrive/KULIAH/PCVK/Images/noises/*.jpg'
noise_files = glob.glob(noises_path_glob)
cv_img = []

if len(noise_files) > 0:
    for img_f in noise_files:
        cv_img.append(cv.imread(img_f))
else:
    print("Folder noises tidak ditemukan. Menggenerasi 100 citra ternoise sintesis...")
    for i in range(100):
        noise = np.random.normal(0, 25, orig_galaxy.shape).astype(np.float64)
        noisy_img = np.clip(orig_galaxy.astype(np.float64) + noise, 0, 255).astype(np.uint8)
        cv_img.append(noisy_img)

# Evaluasi N = 10, 20, 40, 80, 100
n_list = [10, 20, 40, 80, 100]
results = {}

print("=== HASIL EVALUASI PSNR AVERAGE DENOISING ===")
for n in n_list:
    subset = cv_img[:n]
    avg_img = np.clip(np.mean(subset, axis=0), 0, 255).astype(np.uint8)
    psnr_val = calc_PSNR(orig_galaxy, avg_img)
    results[n] = (avg_img, psnr_val)
    print(f"N = {n:3d} Citra | PSNR = {psnr_val:.2f} dB")

fig, axes = plt.subplots(2, 3, figsize=(12, 8))
axes[0, 0].imshow(cv.cvtColor(orig_galaxy, cv.COLOR_BGR2RGB))
axes[0, 0].set_title('Citra Asli (Galaxy)')
axes[0, 0].axis('off')

```

```

idx = 1
for n in n_list:
    r, c = divmod(idx, 3)
    img_res, psnr_v = results[n]
    axes[r, c].imshow(cv.cvtColor(img_res, cv.COLOR_BGR2RGB))
    axes[r, c].set_title(f'N={n} (PSNR: {psnr_v:.2f} dB)')
    axes[r, c].axis('off')
    idx += 1

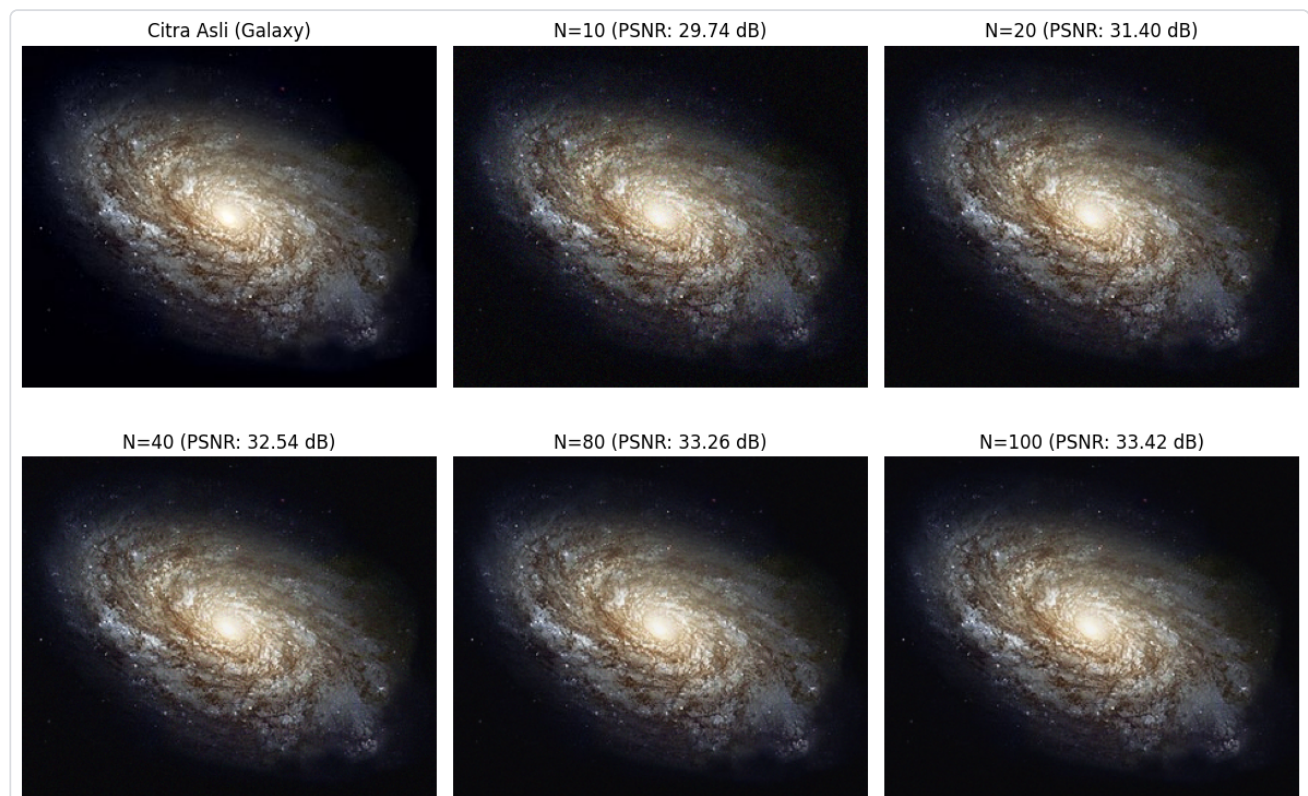
plt.tight_layout()
plt.show()

```

Folder noises tidak ditemukan. Menggenerasi 100 citra ternoise sintesis...

=== HASIL EVALUASI PSNR AVERAGE DENOISING ===

N = 10 Citra | PSNR = 29.74 dB
 N = 20 Citra | PSNR = 31.40 dB
 N = 40 Citra | PSNR = 32.54 dB
 N = 80 Citra | PSNR = 33.26 dB
 N = 100 Citra | PSNR = 33.42 dB



Analisis & Kesimpulan Denoising:

1. Apakah peningkatan jumlah citra selalu memberikan peningkatan PSNR yang signifikan?

Jawab: Tidak selalu secara linear. Peningkatan PSNR terasa sangat drastis pada jumlah citra awal ($N=10$ ke $N=20$). Seiring bertambah besarnya N , kenaikan PSNR mulai melambat (diminishing return) karena varians noise berkurang sebesar faktor $\frac{1}{N}$.

2. Pada jumlah citra berapa peningkatan mulai tidak terlalu signifikan?

Jawab: Peningkatan mulai melambat dan tidak terlalu signifikan setelah $N = 40$ atau $N = 50$.

3. Kesimpulan:

Average Denoising sangat efektif menghapus noise Gaussian bernilai nol-rata-rata. Semakin banyak citra yang dirata-ratakan, semakin tinggi PSNR dan semakin jernih citra hasil.

9. Image Masking (Logical Operators)

Operasi logika citra: **NOT**, **OR**, **AND**, **NAND**, dan **XOR**.

```
img_path = '/content/drive/MyDrive/PCVK/Images/couple.tiff'
try:
    img_couple = load_image(img_path)
except Exception as e:
    print(f"Warning: {e}. Menggunakan citra dummy.")
    img_couple = np.zeros((300, 400, 3), dtype=np.uint8)
    cv.rectangle(img_couple, (50, 50), (150, 250), (180, 100, 100), -1)
    cv.rectangle(img_couple, (220, 50), (320, 250), (100, 180, 100), -1)

h, w, _ = img_couple.shape
mask = np.zeros((h, w), dtype=np.uint8)
cv.circle(mask, (int(w*0.3), int(h*0.3)), 60, 255, -1)
cv.circle(mask, (int(w*0.7), int(h*0.3)), 60, 255, -1)
mask_3ch = cv.cvtColor(mask, cv.COLOR_GRAY2BGR)

res_not = cv.bitwise_not(img_couple)
res_or = cv.bitwise_or(img_couple, mask_3ch)
res_and = cv.bitwise_and(img_couple, mask_3ch)
res_nand = cv.bitwise_not(res_and)
res_xor = cv.bitwise_xor(img_couple, mask_3ch)

fig, axes = plt.subplots(3, 2, figsize=(10, 12))
axes[0, 0].imshow(cv.cvtColor(img_couple, cv.COLOR_BGR2RGB))
axes[0, 0].set_title('Citra Asli')
axes[0, 0].axis('off')

axes[0, 1].imshow(cv.cvtColor(res_not, cv.COLOR_BGR2RGB))
axes[0, 1].set_title('1. NOT (Komplemen)')
axes[0, 1].axis('off')

axes[1, 0].imshow(cv.cvtColor(res_or, cv.COLOR_BGR2RGB))
axes[1, 0].set_title('2. OR')
axes[1, 0].axis('off')

axes[1, 1].imshow(cv.cvtColor(res_and, cv.COLOR_BGR2RGB))
axes[1, 1].set_title('3. AND (Standard Masking)')
axes[1, 1].axis('off')

axes[2, 0].imshow(cv.cvtColor(res_nand, cv.COLOR_BGR2RGB))
axes[2, 0].set_title('4. NAND')
axes[2, 0].axis('off')

axes[2, 1].imshow(cv.cvtColor(res_xor, cv.COLOR_BGR2RGB))
axes[2, 1].set_title('5. XOR')
axes[2, 1].axis('off')
```

```
plt.tight_layout()  
plt.show()
```

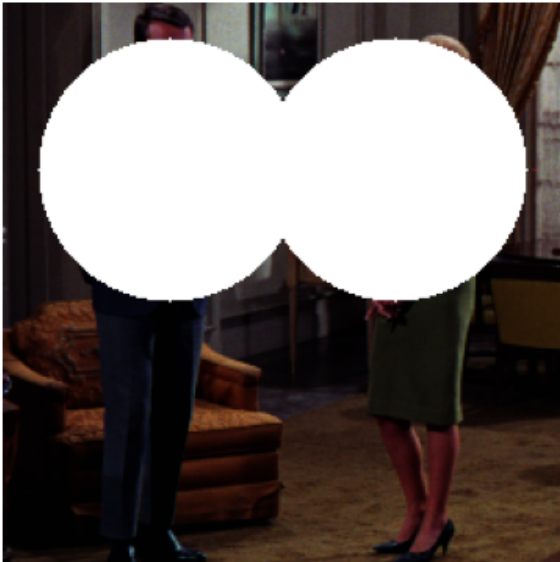

Citra Asli



1. NOT (Komplemen)



2. OR



3. AND (Standard Masking)



4. NAND



5. XOR



Analisis Operator Logika Image Masking:

- **NOT (Komplemen):** Membalikkan nilai intensitas piksel ($255 - x$). Area terang menjadi gelap.
 - **OR:** Menghasilkan nilai maksimum jika salah satu piksel aktif. Piksel di dalam area mask (255/putih) menjadi putih penuh.
 - **AND:** Hanya mempertahankan warna asli piksel pada lokasi mask yang bernilai 255 (aktif). Area di luar mask menjadi hitam (0).
 - **NAND:** Inversi dari hasil AND. Area di dalam mask terinversi, sedangkan area luar mask menjadi putih penuh.
 - **XOR:** Piksel di dalam mask terinversi nilainya, sedangkan piksel di luar mask tetap menampilkan warna citra asli.
-

10. Foto Malam Hari (Night Photo Enhancement)

Metode enhancement untuk foto malam hari yang gelap.

```
img_path = '/content/drive/MyDrive/PCVK/Images/houses.jpg'
try:
    night_img = load_image(img_path)
except Exception as e:
    print(f"Warning: {e}. Menggunakan citra dummy.")
    night_img = np.full((300, 400, 3), 20, dtype=np.uint8)
    cv.rectangle(night_img, (100, 100), (300, 250), (50, 60, 70), -1)

# Gamma Correction dengan gamma < 1 (misal gamma = 0.4)
gamma_val = 0.4
inv_gamma = 1.0 / gamma_val
lut = np.array([((i / 255.0) ** inv_gamma) * 255 for i in np.arange(0, 256)]).astype(np.uint8)
enhanced_night = cv.LUT(night_img, lut)

fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(cv.cvtColor(night_img, cv.COLOR_BGR2RGB))
axes[0].set_title('Foto Malam Asli')
axes[0].axis('off')

axes[1].imshow(cv.cvtColor(enhanced_night, cv.COLOR_BGR2RGB))
axes[1].set_title(f'Gamma Correction ( $\gamma = \{gamma\_val\}$ )')
axes[1].axis('off')
plt.tight_layout()
plt.show()
```

Foto Malam Asli

Gamma Correction ($\gamma = 0.4$)

Analisis Foto Malam Hari:

- **Metode yang Dipilih: Gamma Correction** (dengan $\gamma = 0.4$).
 - **Alasan Pemilihan:** Mengangkat detail piksel berintensitas rendah (area gelap) secara non-linear tanpa membuat area terang mengalami oversaturation.
 - **Resiko Metode:** Jika nilai γ terlalu kecil, noise di area gelap akan ikut diperkuat sehingga citra tampak berbintik-bintik (grainy).
-

11. Perbaikan Kualitas Citra `crayfish.jpg`

Memperbaiki citra `crayfish.jpg` yang pudar/underwater low-contrast.

```
img_path = '/content/drive/MyDrive/PCVK/Images/crayfish.jpg'
try:
    crayfish_orig = load_image(img_path)
except Exception as e:
    print(f"Warning: {e}. Menggunakan citra dummy.")
    crayfish_orig = np.full((300, 400, 3), 110, dtype=np.uint8)
    cv.circle(crayfish_orig, (200, 150), 70, (80, 90, 140), -1)

# 1. Contrast Adjustment (faktor 1.6)
contrast_factor = 1.6
C = (contrast_factor - 1.0) * 128.0
factor = (259.0 * (C + 255.0)) / (255.0 * (259.0 - C))
step1 = np.clip(factor * (crayfish_orig.astype(np.float64) - 128.0) + 128.0, 0, 255).astype(np.uint8)

# 2. Gamma Correction (gamma = 1.2)
gamma_val = 1.2
inv_gamma = 1.0 / gamma_val
lut = np.array([((i / 255.0) ** inv_gamma) * 255 for i in np.arange(0, 256)]).astype(np.uint8)
crayfish_enhanced = cv.LUT(step1, lut)

fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(cv.cvtColor(crayfish_orig, cv.COLOR_BGR2RGB))
axes[0].set_title('Before (crayfish.jpg)')
axes[0].axis('off')

axes[1].imshow(cv.cvtColor(crayfish_enhanced, cv.COLOR_BGR2RGB))
axes[1].set_title('After (Contrast 1.6 + Gamma 1.2)')
axes[1].axis('off')
plt.tight_layout()
plt.show()
```

Before (crayfish.jpg)



After (Contrast 1.6 + Gamma 1.2)



Penjelasan Perbaikan Citra `crayfish.jpg`:

- **Metode yang Dipilih:** Kombinasi **Linear Contrast Enhancement** (faktor 1.6) dan **Gamma Correction** ($\gamma = 1.2$).
- **Alasan Pemilihan:** Citra asli `crayfish.jpg` cenderung memiliki kontras rendah dan warna pudar akibat media air. Penyesuaian kontras merenggangkan intensitas warna agar objek terpisah tegas dari background, sedangkan Gamma Correction menyeimbangkan tingkat kecerahan.
- **Penentuan Parameter Terbaik:** Parameter $C = 1.6$ dan $\gamma = 1.2$ memberikan ketajaman visual optimal tanpa menyebabkan warna terpotong/saturasi berlebih.